

9. Analizador de Rentabilidad de Líneas de Producto — Empresa Distribuidora

Sector: Finanzas / Distribución Comercial | *Empresa distribuidora que exporta su cartera de pedidos a CSV y quiere analizar márgenes por producto y zona*

Descripción del Proyecto

Una empresa distribuidora de bienes de consumo exporta diariamente su cartera de pedidos a un fichero CSV que contiene el detalle de cada línea de pedido: producto, categoría, coste, precio de venta, cantidad, fecha y zona comercial. La dirección financiera necesita una aplicación de escritorio para importar esos datos y calcular KPIs de rentabilidad que hoy se calculan manualmente en Excel.

La aplicación gestiona un catálogo de zonas comerciales y reglas de margen mínimo (CRUD) y, sobre los datos del CSV, calcula el margen bruto por producto y categoría, identifica líneas de producto por debajo del margen mínimo configurado, y muestra la evolución mensual de la facturación y la rentabilidad.

El proyecto desarrolla habilidades de agregación y cálculo financiero básico (margen, EBIT simplificado) sobre grandes colecciones Java, y obliga a distinguir entre datos persistentes (la configuración de negocio) y datos transaccionales (los pedidos del CSV).

Entidades Principales

- **Línea de Pedido (importada desde CSV):** id_línea, id_pedido, referencia_producto, descripción_producto, categoría, coste_unitario, precio_venta_unitario, unidades, fecha_pedido (ISO 8601), zona_comercial, estado (completado/cancelado/pendiente)
- **Zona Comercial (CRUD en app):** ID, nombre, país, responsable comercial, objetivo_facturacion_anual
- **ReglaMargen (CRUD en app):** ID, categoría de producto afectada, margen_minimo_porcentaje, activa, descripción

Requisitos Obligatorios

Requisitos Obligatorios (Must-Have)
• Importar líneas de pedido desde CSV; validar tipos de datos y mostrar filas con error
• CRUD completo de zonas comerciales y reglas de margen mínimo
• Listado de líneas de pedido con filtro por categoría, zona, estado y rango de fechas
• Calcular margen bruto por línea: $(\text{precio_venta} - \text{coste}) \times \text{unidades}$, y margen porcentual
• Ranking de categorías por facturación total ($\text{precio_venta} \times \text{unidades}$) y por margen total
• Evolución mensual de facturación y margen total para la zona o categoría seleccionada
• Detección de líneas con margen por debajo del mínimo de su regla activa
• Comparativa de facturación real vs objetivo por zona comercial
• Validación del CSV: coste y precio positivos, unidades > 0, fecha parseable, estado válido
• Persistencia de zonas y reglas en JSON; las líneas de pedido se cargan en memoria desde CSV

• Exportar ranking de categorías y líneas bajo margen a XLSX con formato de tabla
• GUI con al menos 3 vistas: importación y KPIs globales, explorador de pedidos, panel de rentabilidad
• Manejo de errores: fichero no encontrado, precio o coste negativo, fechas malformadas

Requisitos Nice-to-Have

Mejoras Opcionales (Nice-to-Have)
• Grafica de barras apiladas con facturación vs margen por mes del ultimo año
• Grafica de sectores con la distribución del margen total por categoría
• Proyección de cierre de año: extrapolar facturación anual basándose en la media mensual acumulada
• Sistema de login con roles: comercial (solo su zona) y director financiero (análisis completo)
• Generación de informe ejecutivo de rentabilidad en PDF con KPIs y graficas usando iText
• Integración de IA: identificar las categorías con mayor potencial de mejora de margen y sugerir acciones
• Estructura de código en paquetes: model, view, controller, persistence

Criterios de Proyecto

Definition of Readiness	Definition of Done
• El equipo ha leído y comprendido el enunciado completo del proyecto	• CRUD completo funciona sin errores: crear, consultar, modificar y eliminar
• Se han identificado las entidades del modelo y sus atributos	• Datos guardados automáticamente y recuperados al iniciar la aplicación
• Se ha acordado el formato de persistencia a usar (JSON, CSV, XLSX, etc.)	• Interfaz gráfica (Swing o JavaFX) completamente funcional
• Entorno configurado: JDK, IDE (IntelliJ/Eclipse), Maven/Gradle	• Búsqueda o filtrado de registros funciona correctamente
• Dependencias (librerías) identificadas y descargadas: Jackson, Apache POI, JavaFX, etc.	• Todas las validaciones implementadas con mensajes de error claros al usuario
• Estructura de paquetes creada: model/, view/, controller/, persistence/	• Exportación a archivo (CSV/XLSX/PDF) genera el fichero correcto
• Planificación inicial realizada y trabajo repartido entre los miembros del equipo	• Código organizado en los paquetes definidos (model/view/controller/persistence)
• No existen dependencias técnicas bloqueantes sin resolver	• Errores comunes gestionados sin cierres inesperados del programa

	• La aplicación tiene al menos 3 vistas o pantallas diferenciadas
	• Proyecto probado manualmente con casos de uso reales por el equipo